



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

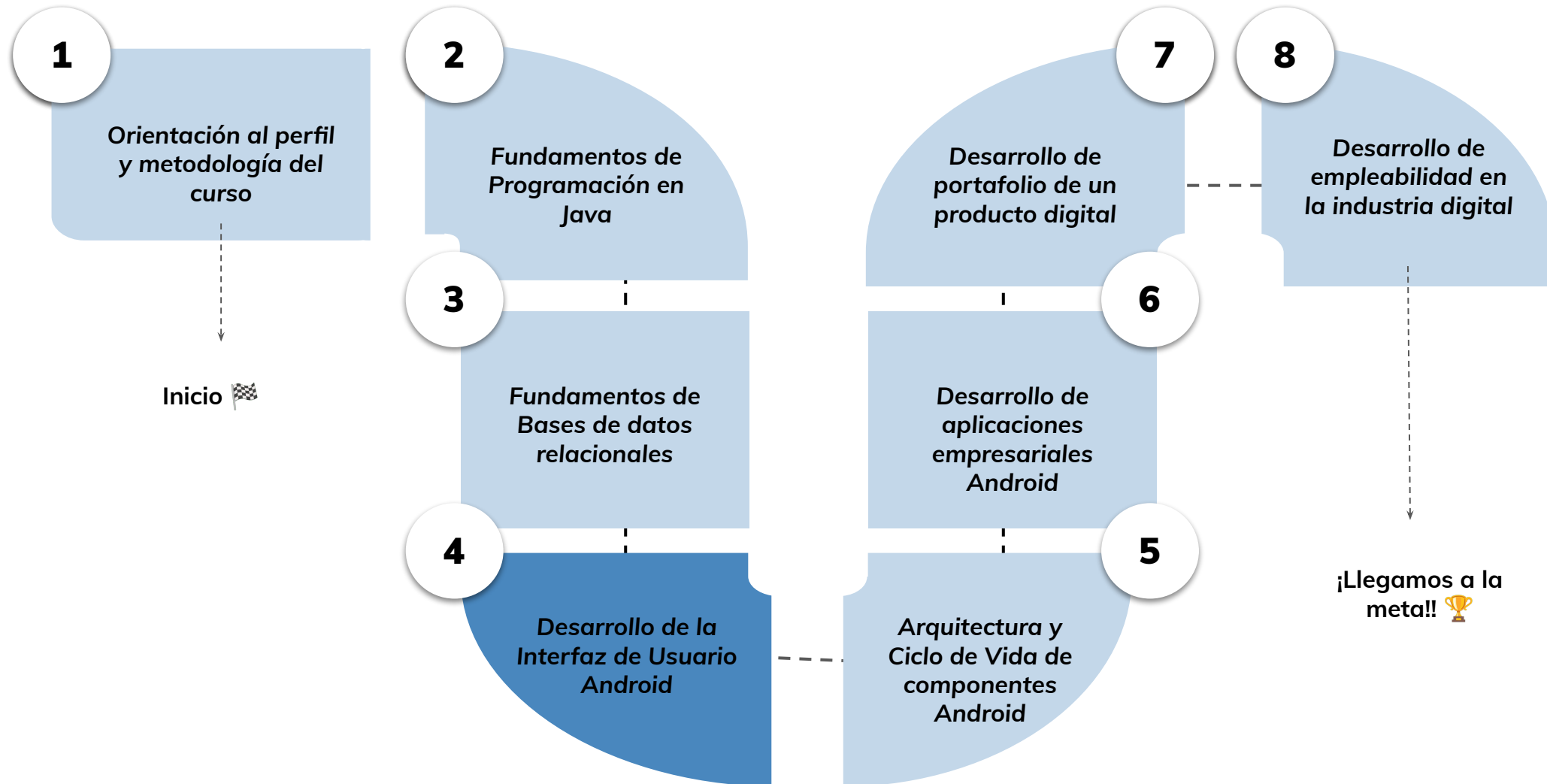


› Elementos de configuración de un proyecto android

AE3.1: Distinguir elementos de configuración de un proyecto android distinguiendo mecanismos para el manejo de assets para el desarrollo de aplicaciones nativas.

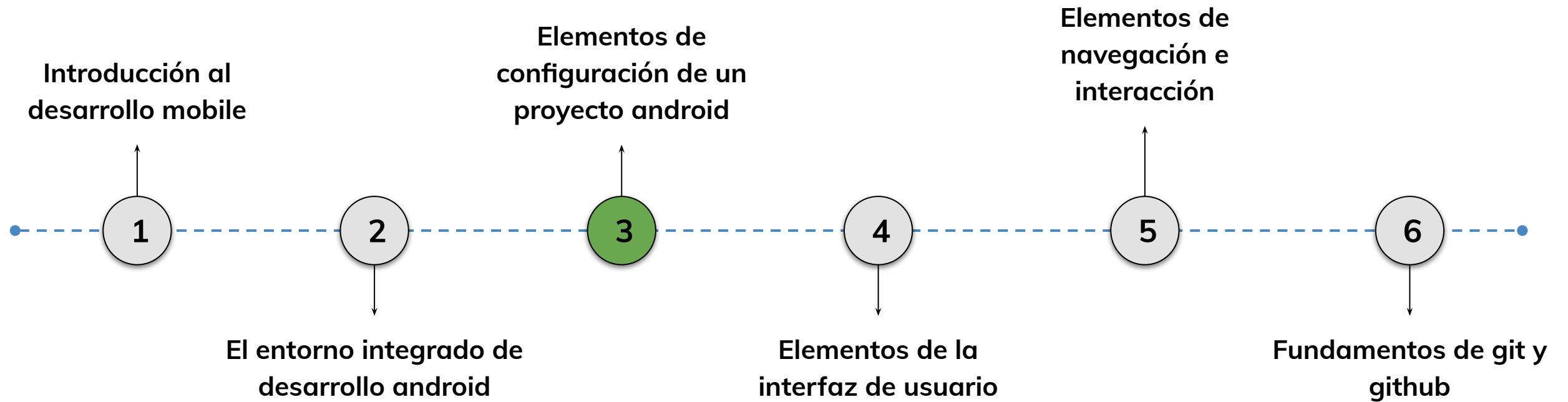
Hoja de ruta

¿Cuáles **módulos** conforman el programa?



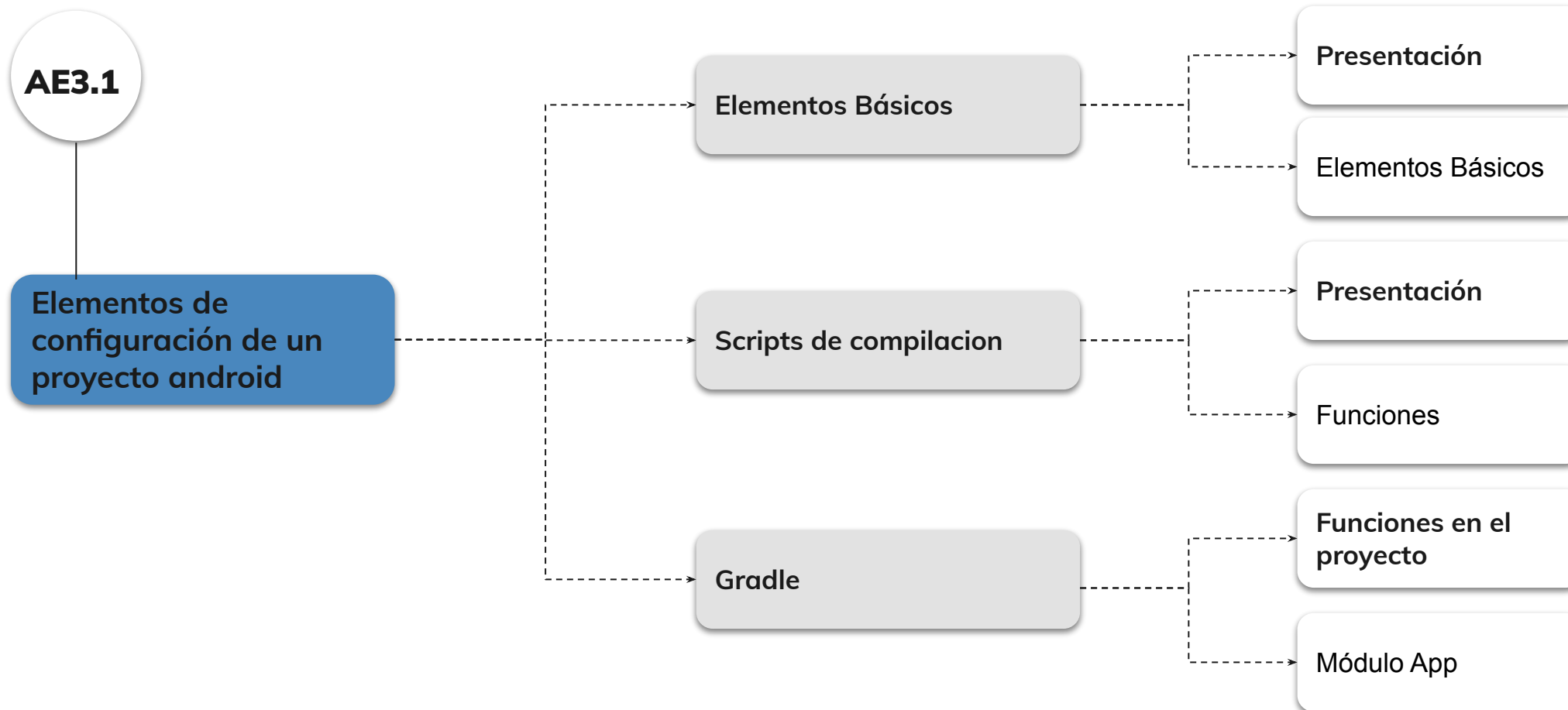
Roadmap de lecciones

¿Cuáles lecciones estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?





Objetivos de aprendizaje

¿Qué aprenderás?



- Conocer los elementos de un proyecto Android
- Entender las principales funciones de los scripts de compilación
- Ver cómo se maneja Gradle del module app

< > Rompehielo

Contexto:

Al crear un nuevo proyecto en Android Studio se crean varias carpetas y archivos que nos dan una base a partir de la cual nosotros construimos nuestras aplicaciones. Varios de ellos son parte de las mejores prácticas de Android también.

Consigna:

Si observamos con detalle la carpeta res (resources/recursos) vamos a encontrar un subdirectorio llamado values (valores) donde están los archivos “strings” y “colors”

¿Para qué piensan que sirven estos 2 archivos en un proyecto?



Elementos Básicos



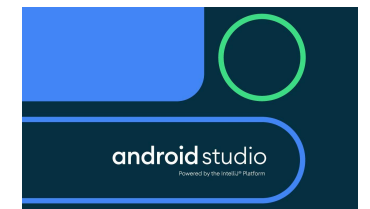
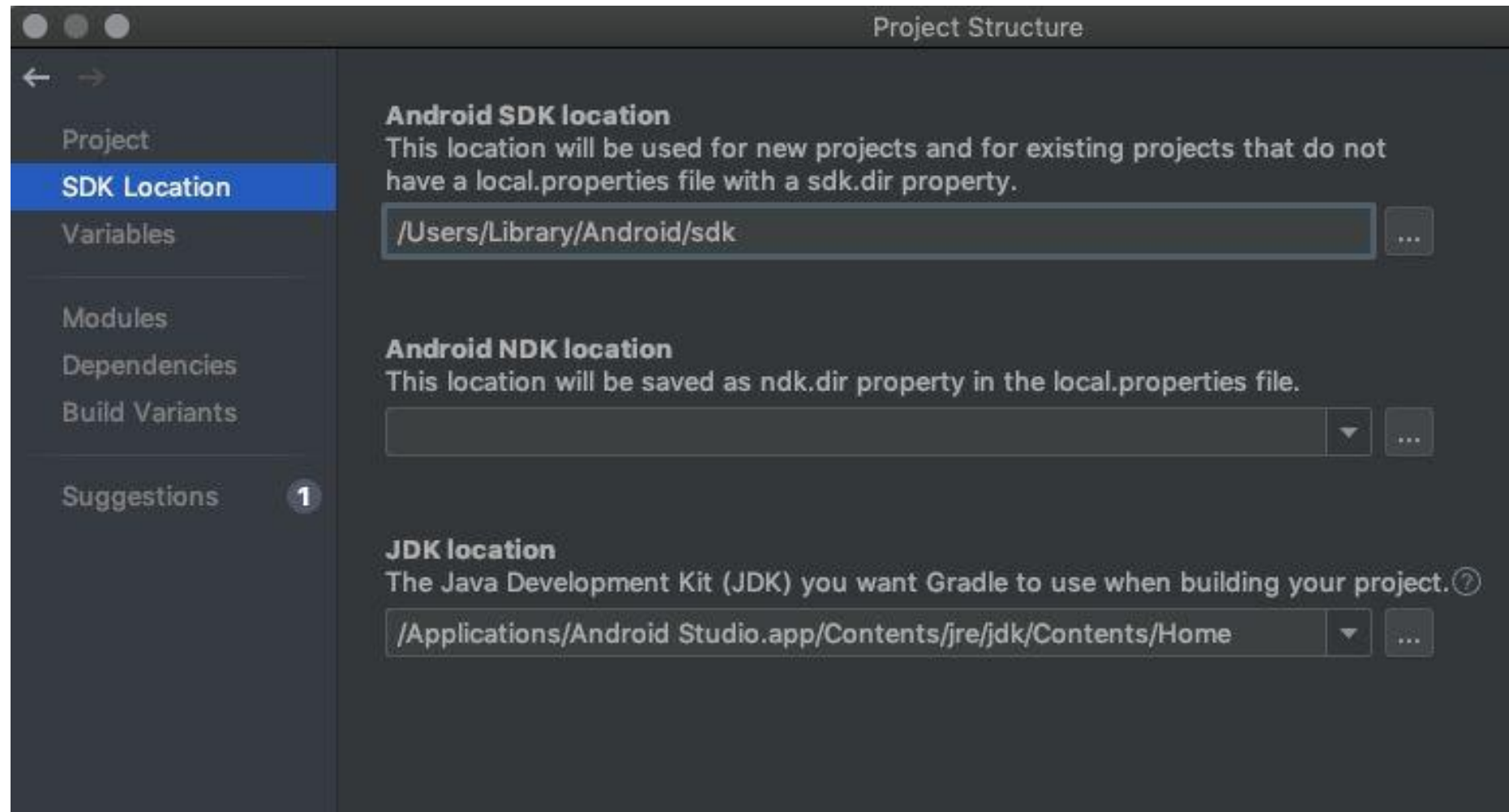
Elementos básicos



¿Qué son?:

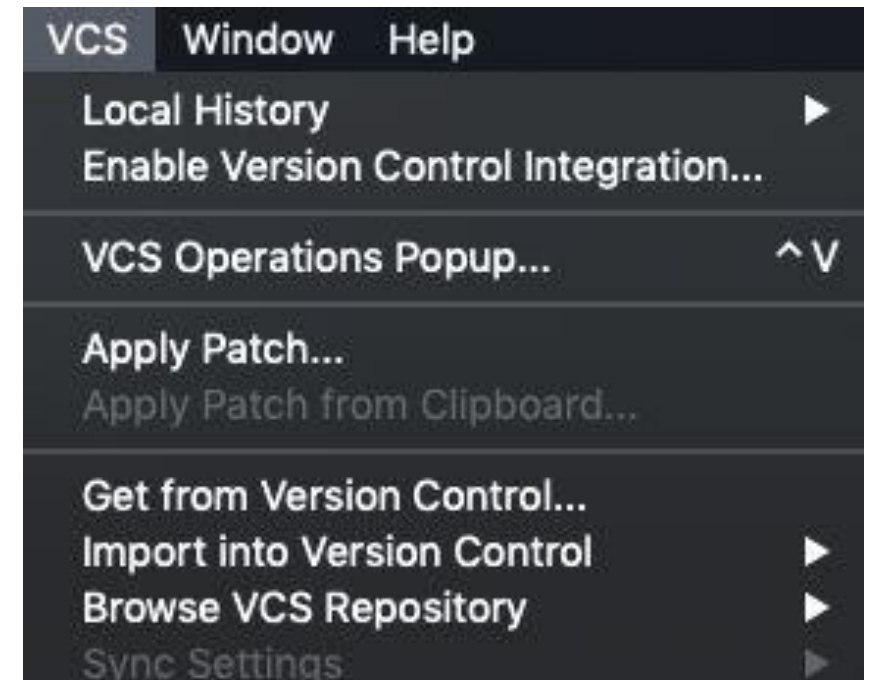
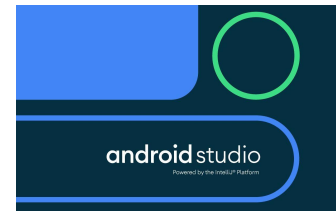
Android nos proporciona con muchas herramientas y estructuras que nos ayudarán en el desarrollo de nuestras aplicaciones. Algunas son directorios que facilitan el agregado de recursos y mantenerlos ordenados y otros son grupos de archivos que nos permiten setear valores y opciones para toda la aplicación

SDK Location



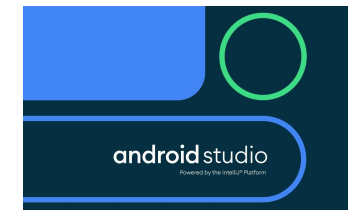
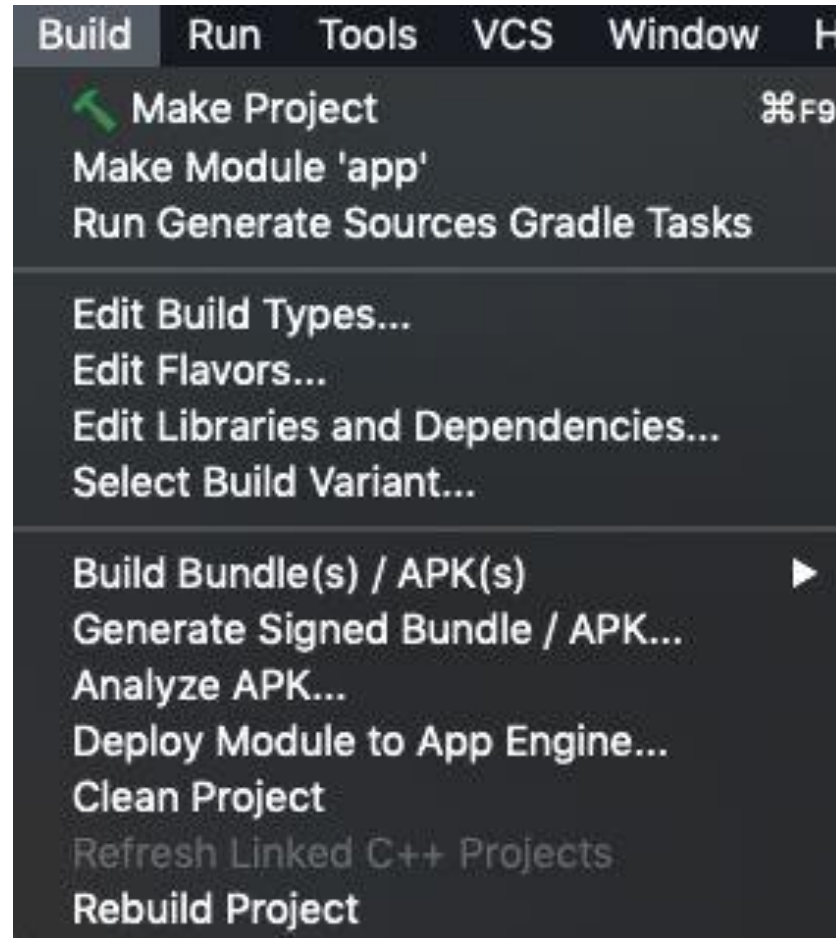
Control de versiones

- Enable Version Control Integration
- Get from Version Control



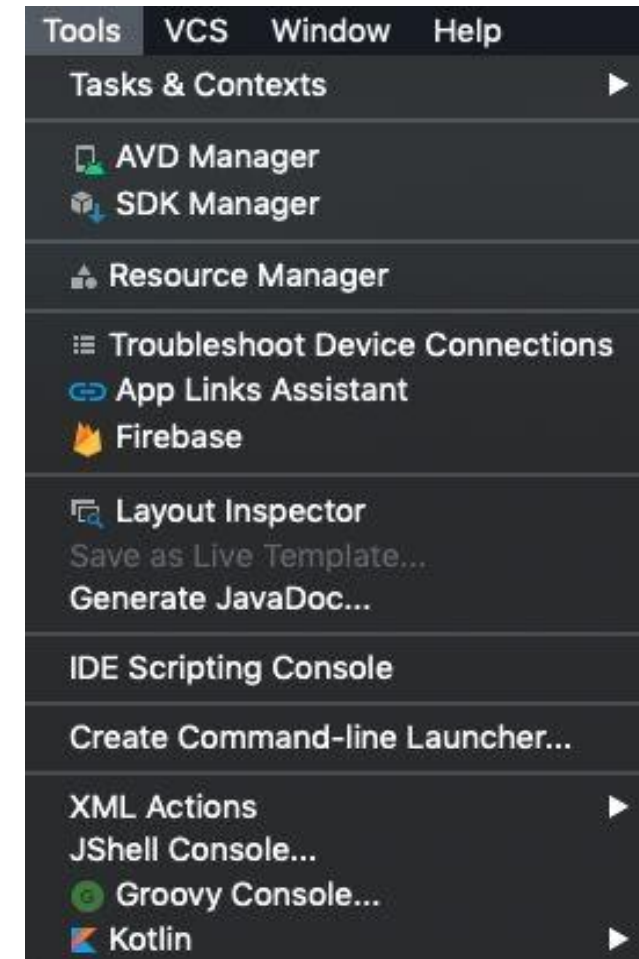
Build

1. Make Project
2. Clean Project
3. Rebuild Project



Tools

- [Android Virtual Devices Manager](#)
- [Software Development Kit Manager](#)
- [Resource Manager](#)
- [Firebase](#)
- [Layout Inspector](#)



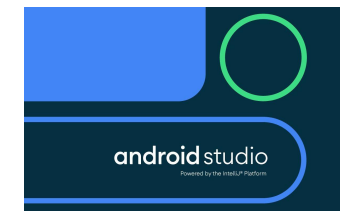
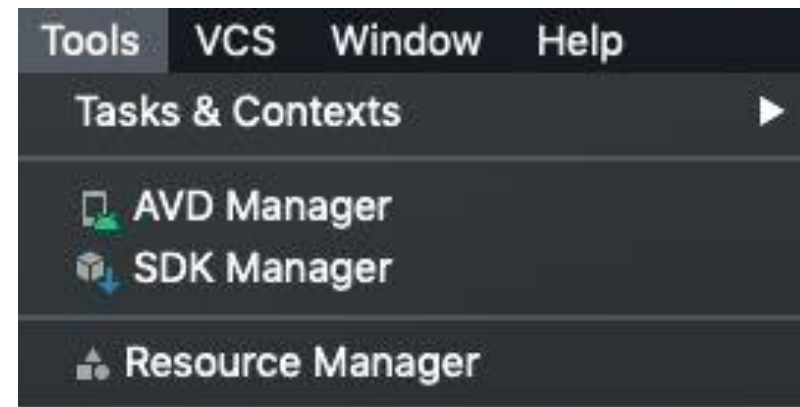
Ejercicio: Nuestra primera App

- Crear una nueva app (File → New → New Project...)
- Elegir el template Empty Activity
- Asignar el nombre
- Configurar package y ubicación
- Configurar nivel de API
- Compilar la app
- Probar en un dispositivo



Probar la app

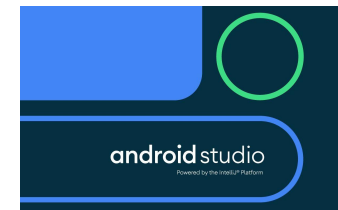
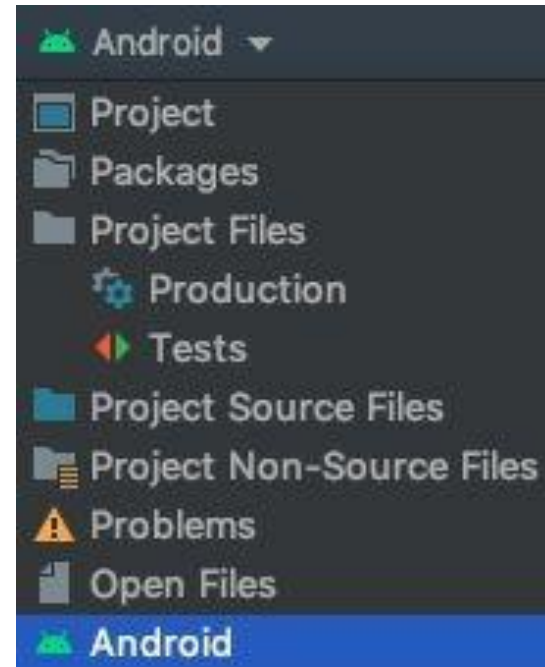
- Teléfono usando cable USB
- Dispositivos virtuales
 - [AVD Manager](#)
 - [Genymotion](#)



Estructura de directorios y vista de un proyecto

Vistas del proyecto

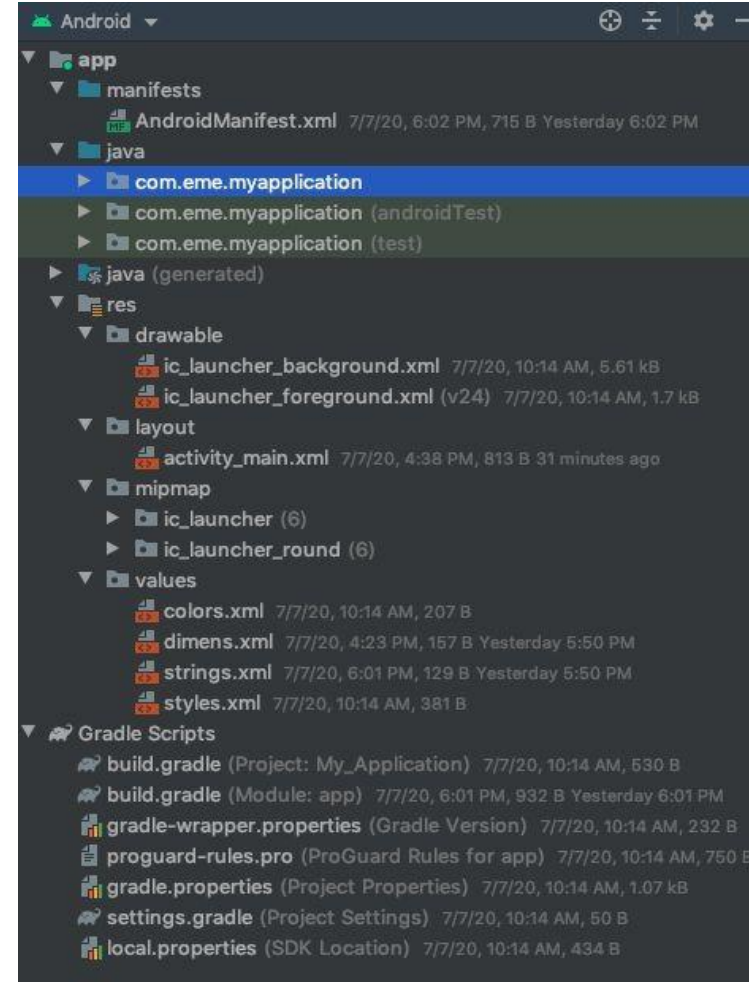
- Android
- Project



Vistas Android

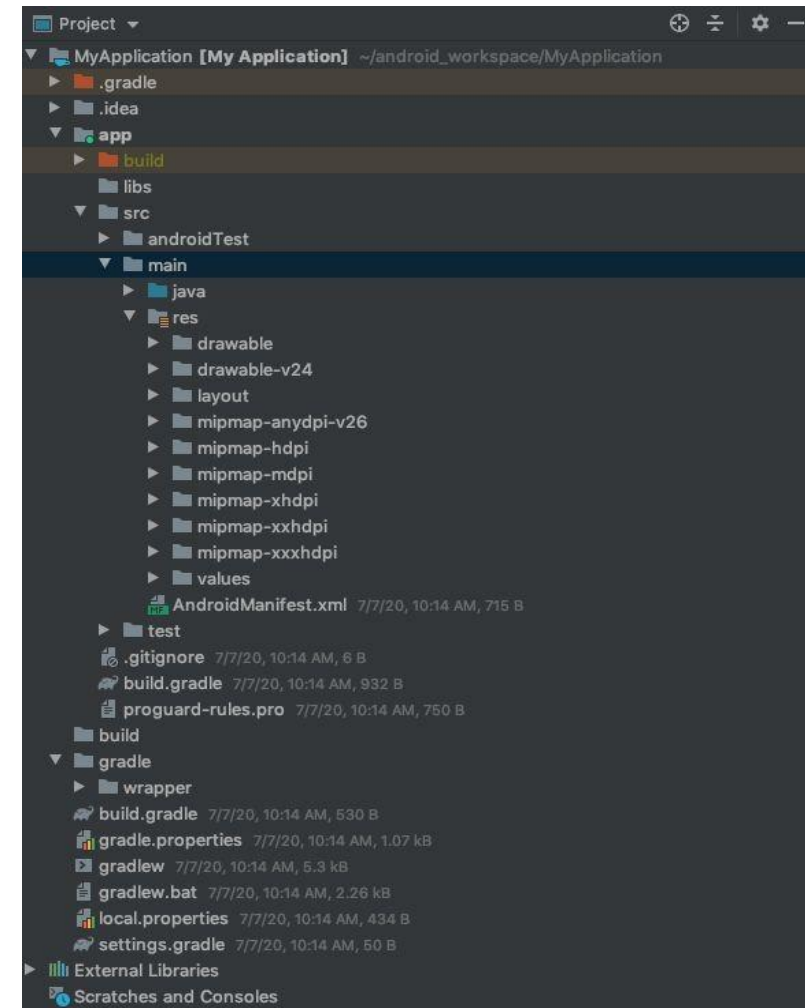
- app
 - manifests
 - java
 - res

- Gradle Scripts



Estructura de directorios: App

- build/
- libs/
- src/
 - androidTest/
 - main/
 - AndroidManifest.xml
 - java/
 - res/
 - values/
 - colors.xml
 - dims.xml
 - strings.xml
 - styles.xml
- test/



Densidad de Pantalla

Tabla 1: Calificadores de configuración para diferentes densidades de píxeles

Calificador de densidad	Descripción
ldpi	Recursos para pantallas de densidad baja (<i>ldpi</i>) (120 dpi)
mdpi	Recursos para pantallas de densidad media (<i>mdpi</i>) (~160 dpi; esta es la densidad de referencia)
hdpi	Recursos para pantallas de densidad alta (<i>hdpi</i>) (~240 dpi)
xhdpi	Recursos para pantallas de densidad muy alta (<i>xhdpi</i>) (~320 dpi)
xxhdpi	Recursos para pantallas de densidad muy, muy alta (<i>xxhdpi</i>) (~480 dpi)
xxxhdpi	Recursos para usos de densidad extremadamente alta (<i>xxxhdpi</i>) (~640 dpi)
nodpi	Recursos para todas las densidades. Estos son recursos independientes de la densidad. El sistema no escala recursos etiquetados con este calificador, independientemente de la densidad de la pantalla actual.
tvdpi	Recursos para pantallas entre mdpi y hdpi; de aproximadamente 213 dpi. Este no se considera un grupo de densidad "principal". Se usa más que nada para televisiones, y la mayoría de las apps no lo necesitarán ya que con recursos mdpi y hdpi será suficiente; el sistema los escalará según corresponda. Si crees que es necesario proporcionar recursos tvdpi, debes ajustar su tamaño con un factor de 1.33*mdpi. Por ejemplo, una imagen de 100 x 100 píxeles para pantallas mdpi debe ser de 133 x 133 píxeles para tvdpi.

- 36 x 36 (0.75x) para densidad baja (ldpi)
- 48 x 48 (referencia de 1.0x) para densidad media (mdpi)
- 72 x 72 (1.5x) para densidad alta (hdpi)
- 96 x 96 (2.0x) para densidad muy alta (xhdpi)
- 144 x 144 (3.0x) para densidad muy, muy alta (xxhdpi)
- 192 x 192 (4.0x) para densidad extremadamente alta (xxxhdpi)

XML

- Es un **lenguaje de etiquetas** desarrollado por el World Wide Web Consortium (W3C)
- Se enfoca en el sentido semántico de los datos dejando de la lado la presentación
- Se utiliza para almacenar datos en forma legible para humanos

```
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>
    <text>Don't forget
me!</text>
    <alarm>Sunday 23,
06:00</alarm>
  </body>
</note>
```

Tipos de recursos

Los recursos son archivos adicionales con contenido estático que se almacenan en el directorio res/. En tiempo de ejecución, es Android que hace uso de los recursos apropiados basado en la configuración actual del dispositivo

Algunos de los [tipos de recursos](#) soportados son:

- Animación
- Lista de estados de colores
- Drawable (bitmaps o XML)
- Layout
- Menú
- String
- Style
- Font



developer.android.com



res/values/colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <color name="colorPrimary">#6200EE</color>
  <color name="colorPrimaryDark">#3700B3</color>
  <color name="colorAccent">#03DAC5</color>
</resources>
```



res/values/dimens.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <dimen name="margin_normal">16dp</dimen>
  <dimen name="margin_small">8dp</dimen>
  <dimen name="padding_default">7dp</dimen>
</resources>
```



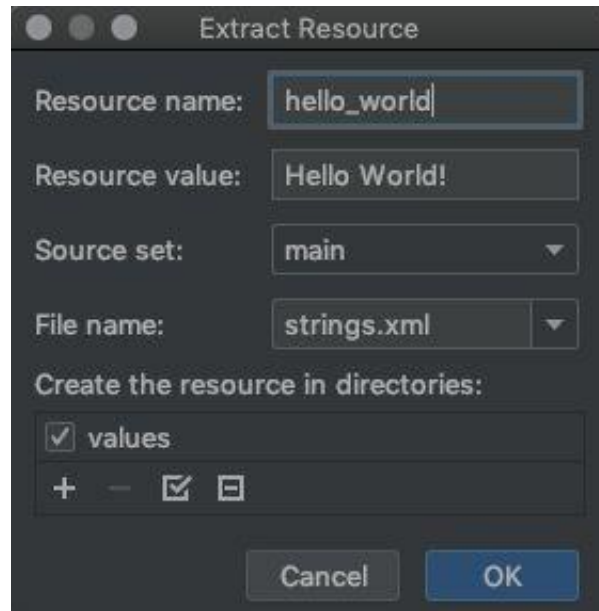
Internacionalización

Android permite correr una app en todo el mundo. Al usar archivos que permitan [localization](#), la información entregada a los usuarios depende de su configuración Strings.xml (botón derecho) → Open Translations Editor

```
<resources>
  <string name="app_name">AndroidFundations</string>
  <string name="to">to</string>
  <string name="subject">subject</string>
  <string name="message">message</string>
  <string name="send">send</string>
  <string name="button_3">Third Button</string>
  <string name="button_2">Second Button</string>
  <string name="button_1">First Button</string>
</resources>
```

```
<resources>
  <string name="app_name">FundamentosAndroid</string>
  <string name="to">para</string>
  <string name="subject">asunto</string>
  <string name="message">mensaje</string>
  <string name="send">enviar</string>
  <string name="button_3">Tercer botón</string>
  <string name="button_2">Segundo botón</string>
  <string name="button_1">Tercer botón</string>
</resources>
```


Utilizar recursos



```
<resources>
  <string name="app_name">My Application</string>
  <string name="hello_world">Hello World!</string>
</resources>
```

```
<TextView
  android:id="@+id/tvMain"
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:text="Hello World!"
  android:textSize="30sp"
  app:layout_constraintBottom_toBottomOf="parent"
  app:layout_constraintLeft_toLeftOf="parent"
  app:layout_constraintRight_toRightOf="parent"
  app:layout_constraintTop_toTopOf="parent" />
```

```
<TextView
  android:id="@+id/tvMain"
  android:layout_width="wrap_content"
  android:layout_height="wrap_content"
  android:text="@string/hello_world"
  android:textSize="@dimen/normal_text_size"
  app:layout_constraintBottom_toBottomOf="parent"
  app:layout_constraintLeft_toLeftOf="parent"
  app:layout_constraintRight_toRightOf="parent"
  app:layout_constraintTop_toTopOf="parent" />
```



Momento:

Time-out!



5 -10 min.



¿Cómo se construye una APP?



Gradle



- Es un sistema de automatización de construcción
- Es de código abierto
- Construído sobre los conceptos de Ant y Maven
- Introduce a [Groovy](#) como [DSL](#) por sobre [XML](#)
- Utiliza un Grafo Acíclico Dirigido ([DAG](#)) para determinar el orden en que las tareas pueden ser ejecutadas
- Diseñado para construcciones multi-proyecto





Android Gradle Plugin



- El [Android Gradle Plugin](#) agrega funcionalidades propias del desarrollo Android
- Se actualiza en conjunto con el IDE (aunque pueden ser independientes)
- La versión actual es la 7.1.0 (Junio 2023)
- El archivo build.gradle (Project) para configuraciones comunes a todos los sub-proyectos / módulos
- El archivo **build.gradle (Module:app)** es para la configuración del módulo por defecto **app**





Build.gradle(Module:app)

- compileSdkVersion
- minSdkVersion
- targetSdkVersion
- Tipos de construcción
- Dependencias

<https://developer.android.com/guide/topics/manifest/uses-sdk-element>

```
apply plugin: 'com.android.application'

android {
    compileSdkVersion 29
    buildToolsVersion "29.0.3"

    defaultConfig {
        applicationId "com.eme.android.java"
        minSdkVersion 24
        targetSdkVersion 29
        versionCode 1
        versionName "1.0"
    }

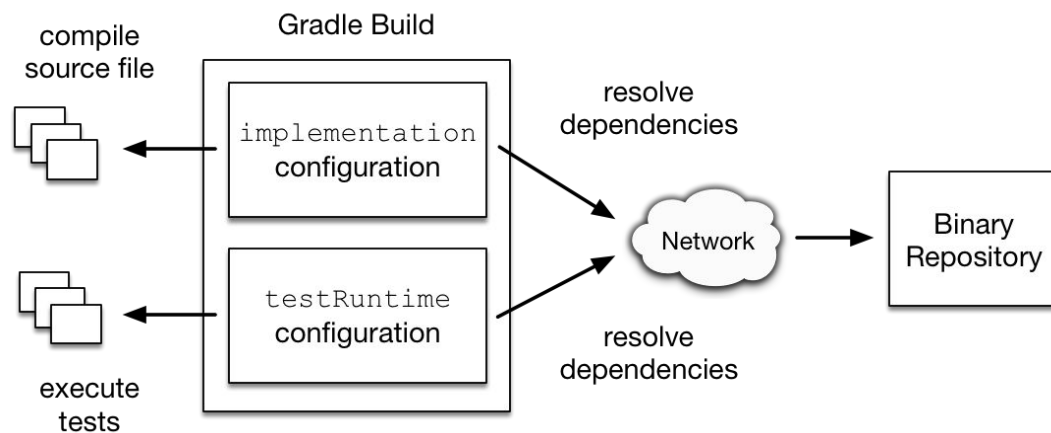
    buildTypes {
        release {
            minifyEnabled false
            proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'),
                'proguard-rules.pro'
        }
    }
}

dependencies {
    implementation fileTree(dir: "libs", include: ["*.jar"])
    implementation 'androidx.appcompat:appcompat:1.1.0'
    ---
}
```

Dependencias

- implementation
- testImplementation
- androidTestImplementation

```
dependencies {  
    implementation fileTree(dir: "libs", include: ["*.jar"])  
    implementation 'androidx.appcompat:appcompat:1.1.0'  
    implementation 'androidx.constraintlayout:constraintlayout:1.1.3'  
    implementation 'com.google.android.material:material:1.1.0'  
  
    testImplementation 'junit:junit:4.12'  
  
    androidTestImplementation 'androidx.test.ext:junit:1.1.1'  
    androidTestImplementation 'androidx.test.espresso:espresso-core:3.2.0'  
}
```



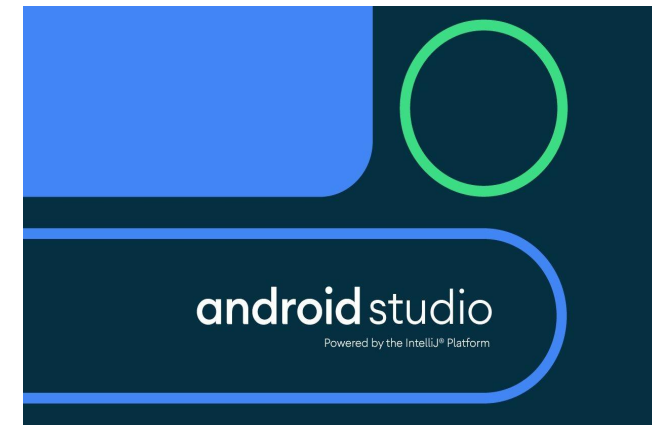
Gradle files have changed since last project sync. A project sync may be necessary for the IDE to work properly.

[Sync Now](#)

Estructura del proyecto

File > Project Structure...

- Configuración del SDK
- **Gestión de dependencias**
- Configuraciones de construcción de app
- Tipos de construcción





Manifest

- El [manifiesto](#) se ubica en la raíz del código del proyecto
- Debe llamarse **AndroidManifest.xml**
- Describe información esencial de la app para el SO y Google Play
 - Nombra el paquete de la app
 - Describe los componentes de la app
 - Declara los permisos que pide la app para funcionar

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.eme.android.java">

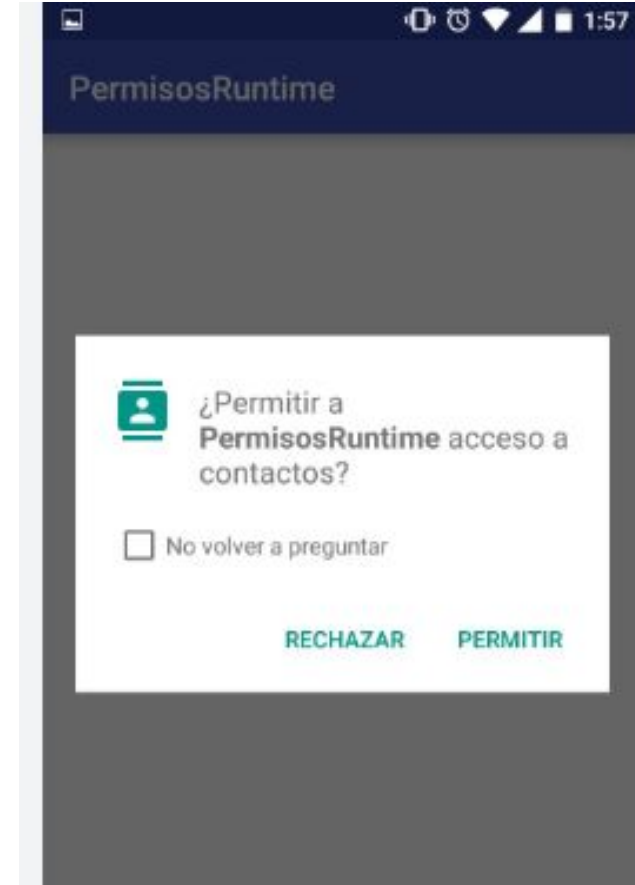
    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportRtl="true"
        android:theme="@style/AppTheme">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

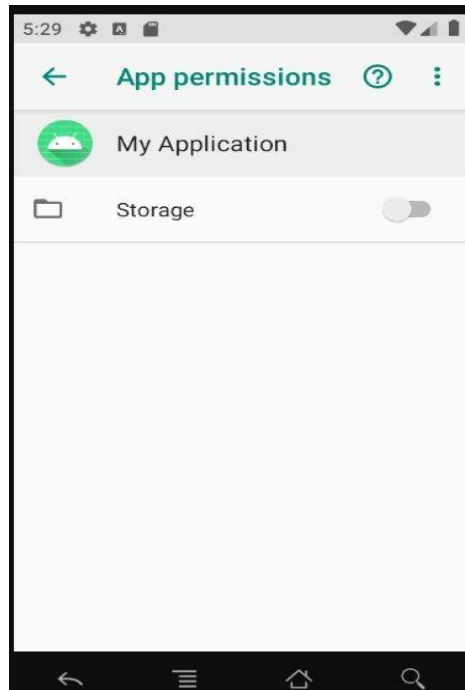
Permisos

- Su propósito es proteger la privacidad del usuario
- Por defecto las apps no pueden realizar operaciones que impacten otras apps, al sistema operativo o al usuario
- La clase [Manifest.permission](#) define la lista de permisos disponibles



Declarar Permisos

Usar el tag `<uses-permission>` para exponer los permisos requeridos



```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.eme.android.java">

    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />

                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>

</manifest>
```

Ejercicio de dispositivos virtuales

Vamos a crear:

- Un teléfono
- Pantalla de 5"
- API level 31

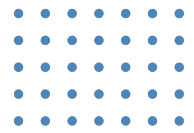
Vamos a correr la app recientemente creada en ese dispositivo





Ejercicio N° 1

Calculadora de String



Calculadora de String

Consigna: ✍️

1. **Método:** Implementar un método que reciba un **String** con números enteros separados por comas.
2. **Entrada:** El método recibe un **String** que contiene números enteros separados por comas (por ejemplo, "1,2,3").
3. **Salida:** El método debe retornar la suma de los números enteros.
 - Si el **String** está vacío, el resultado debe ser 0.
 - Si el **String** contiene un solo número, el resultado debe ser ese número.
 - Si el **String** contiene varios números, debe retornar la suma de esos números.



Cómo asignar el resultado en pantalla



Desde el layout, identificar el nombre del ID de la vista (widget)

Para `android:id="@+id/textViewMain"`

El nombre es: `textViewMain`

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
...
>

    <TextView
        android:id="@+id/textViewMain"
        android:text="Hello World!"
        ...
    />
</androidx.constraintlayout.widget.ConstraintLayout>
```

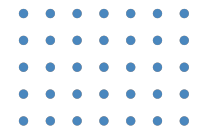



Cómo asignar el resultado en pantalla

En MainActivity, dentro del método onCreate()

Usamos findViewById() para enlazar la vista

```
public class MainActivity extends AppCompatActivity {  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
  
        // Buscar y enlazar la vista  
        TextView textView = findViewById(R.id.textViewMain);  
  
        int result = 1;  
        textView.setText("El resultado es " + result);  
    }  
}
```

Cómo asignar el resultado en pantalla



```
public class SumaString {

    public static int sumarNumeros(String input) {
        if (input.isEmpty()) {
            return 0;
        }

        String[] numeros = input.split(",");
        int suma = 0;

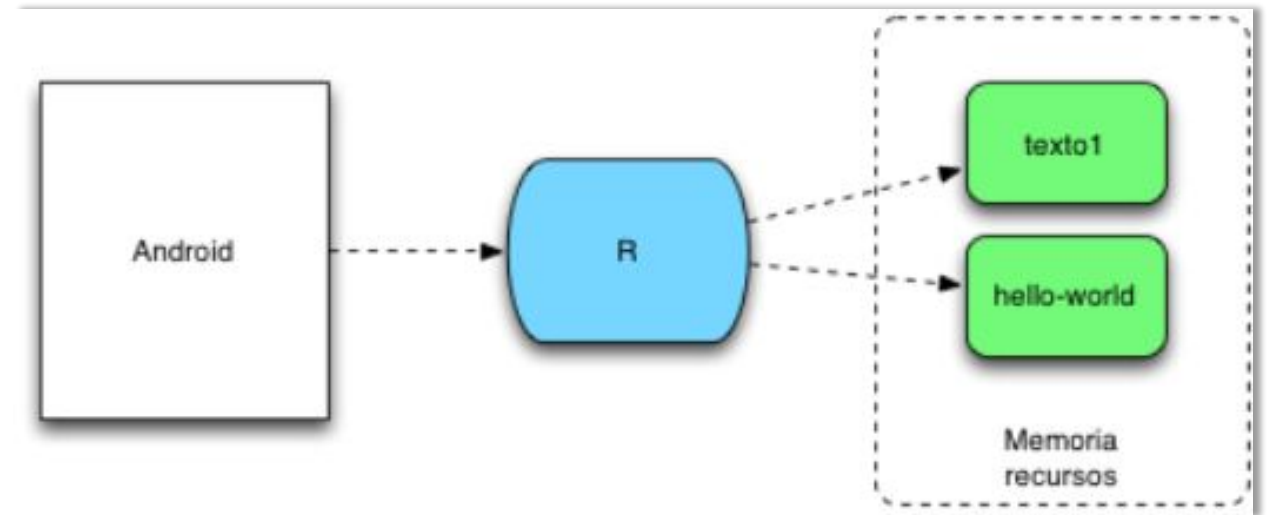
        for (String num : numeros) {
            suma += Integer.parseInt(num);
        }

        return suma;
    }

    public static void main(String[] args) {
        System.out.println(sumarNumeros(""));           // 0
        System.out.println(sumarNumeros("1"));          // 1
        System.out.println(sumarNumeros("1,2"));        // 3
        System.out.println(sumarNumeros("3,5,7"));      // 15
    }
}
```

Clase R

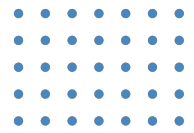
- La [clase R](#) es generada dinámicamente
- Creada durante el proceso de compilación
- Identifica todos los assets (textos, botones, widgets) para usarlas en las clases Java
- Contiene constantes públicas para identificar los assets





Ejercicio N° 2

Calculadora de String



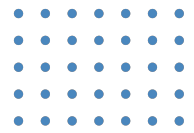
Calculadora de String

Consigna:

1. [opcional] Escribir la clase **Calculadora**
2. Crear el método **add** que recibe un String con los números que vienen separados por coma y retorna la suma.

Por ejemplo, puede recibir "" o "1" o "1,2".El resultado para el String vacío es 0 por definición

3. El resultado debe ser mostrado en pantalla
4. [opcional] Permitir que el método add pueda manejar una cantidad desconocida de números
5. [opcional] Soportar distintos tipos de delimitadores

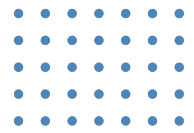


Método add

- Definir el método add
- Imprimir el resultado en el método onCreate()
- Modificar el texto del textView con el resultado

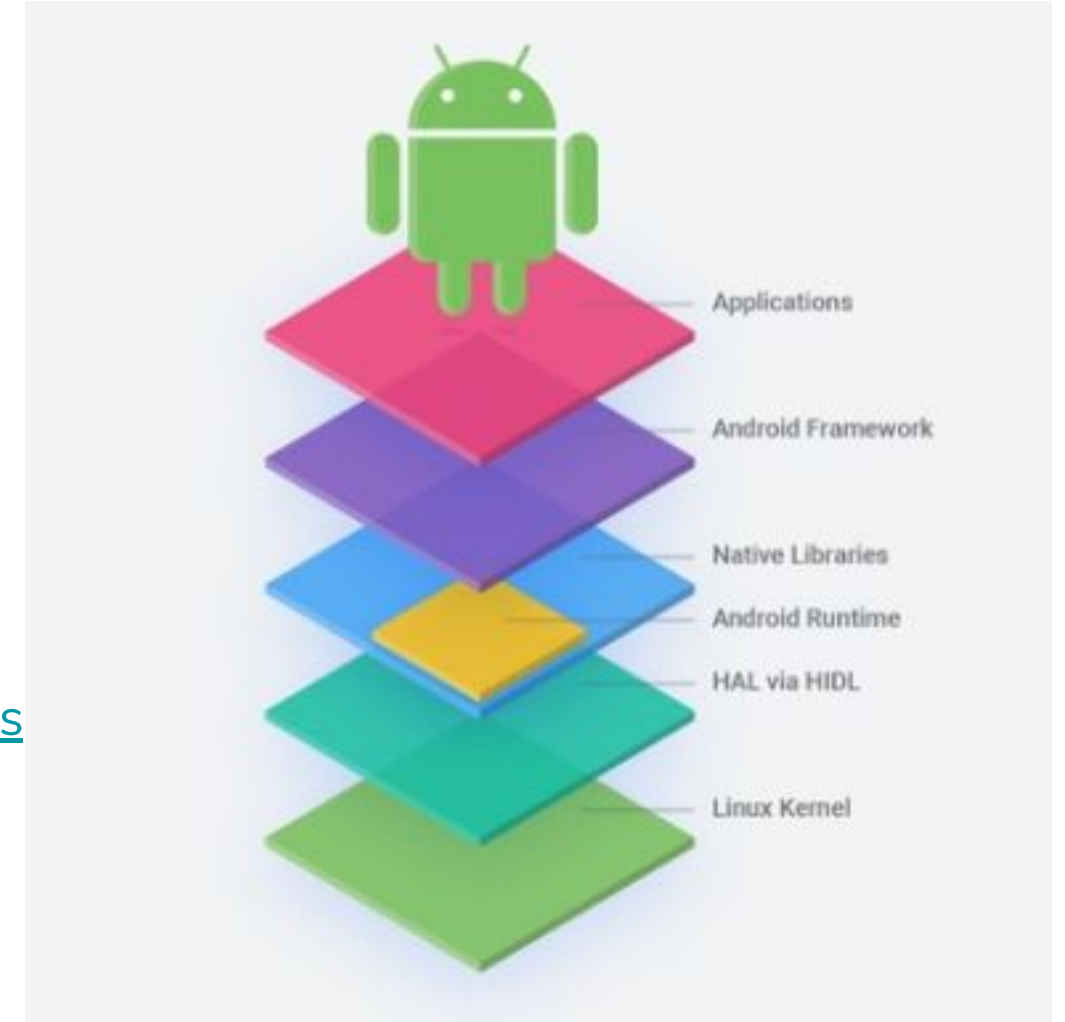
Para separar el String se puede utilizar [String.split\(\)](#) o [StringTokenizer](#)

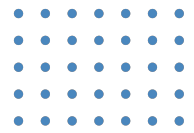
```
public class MainActivity extends AppCompatActivity {  
  
    private static final String TAG = "MainActivity";  
  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
        TextView textView = findViewById(R.id.textViewMain);  
  
        int result = add( "" );  
        Log.d(TAG, "El resultado es " + result);  
        textView.setText("El resultado es " + result);  
    }  
  
    private int add( String chain ) {  
        if ( chain.isEmpty() ) {  
            return 0;  
        }  
  
        return -1;  
    }  
}
```



Extra: Recursos de Android

- Documentación oficial
 - <https://developer.android.com>
- Codelabs
 - <https://codelabs.developers.google.com/android>
- Google Developers Group (GDG)
 - <https://www.meetup.com/es-ES/gdg-chile/>
- Canal de youtube
 - <https://www.youtube.com/user/androiddevelopers>
- Google IO
 - <https://events.google.com/io/>

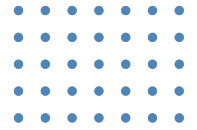




Assets

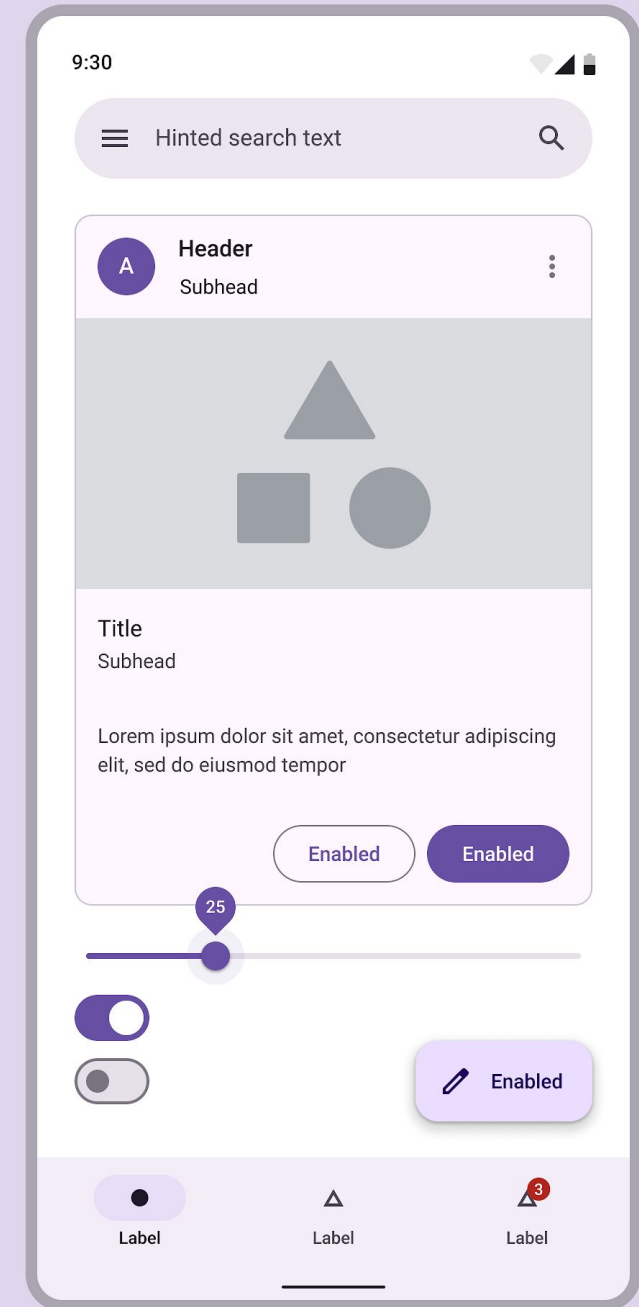
Los archivos llamados Assets son recursos esenciales en nuestros proyectos de aplicación. Estos pueden incluir imágenes, archivos de audio, texto, HTML, fuentes personalizadas y más. A diferencia de otros archivos, como los de diseño, los archivos en la carpeta Assets no son procesados por el sistema de compilación de Android, lo que les permite mantener su formato original y luego ser empaquetados en la aplicación. Estos archivos se encuentran en una carpeta llamada "Assets" dentro del directorio del proyecto y ofrecen la ventaja de ser accesibles directamente desde el código de la aplicación durante la ejecución, lo que significa que pueden cargarse según sea necesario en la aplicación.

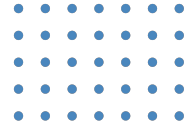
```
lsVideo = ( (type == "image" || type == "video") ?  
lsUrl = ( source.indexOf("youtube.com") > -1 ?  
lsElement = ( (type == "url" || type == "image") ?  
lsObject = ( (typeof subject == "string" ||  
  
check if boxer is already active, return  
$("#boxer").length > 1 || (typeof image  
return;  
  
all event  
Event(e);  
  
cache internal data  
= $.extend({}, {  
$window: $(window),  
$body: $("body"),  
$target: $target,  
$object: $object,  
visible: false,  
resizeTimer: null,  
touchTimer: null,  
gallery: {  
  active: false  
}
```

Styles

En el desarrollo de aplicaciones, los estilos (Styles) son conjuntos de atributos predefinidos que permiten diseñar la apariencia de un proyecto. Estos estilos, que incluyen elementos como fondos, colores, tamaños de fuente, márgenes y bordes, son esenciales para dar a cada aplicación su identidad visual única. Utilizar estilos predefinidos o crearlos al inicio del proyecto ahorra tiempo al evitar tener que escribir código repetitivo, como el diseño de botones. Es importante mantener la coherencia al utilizar los colores y diseños definidos a lo largo del proyecto una vez que se han establecido.





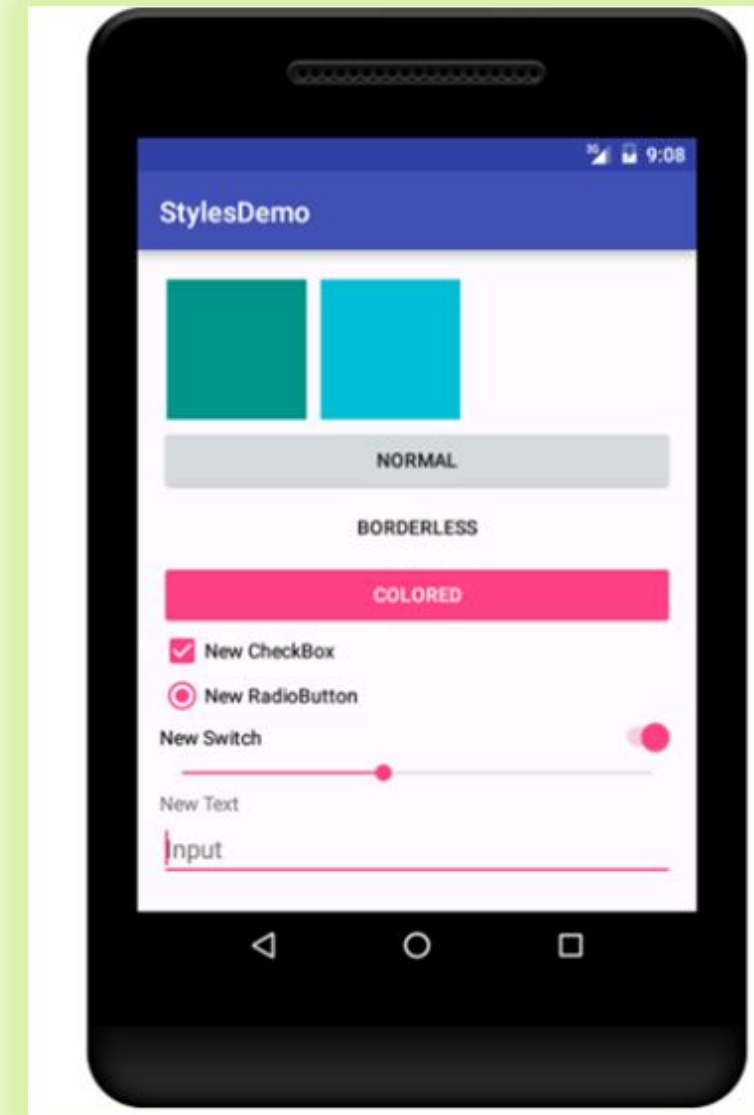
Res/values/Style.xml

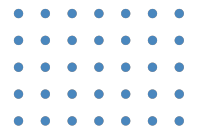
Estilo (Style)

Colección de atributos que especifican la apariencia de una vista (widget)

Tema (Theme)

Es un tipo de estilo que se aplica a una app entera, actividad o una estructura de vistas

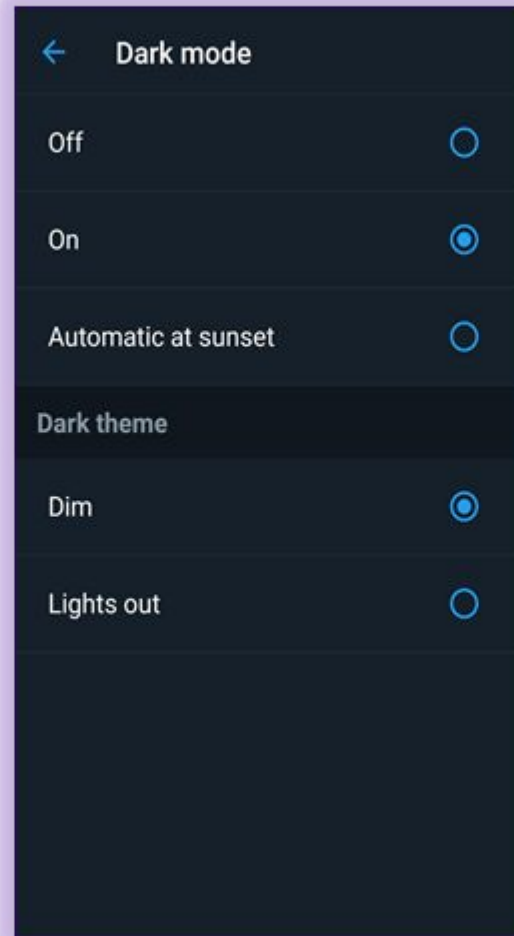
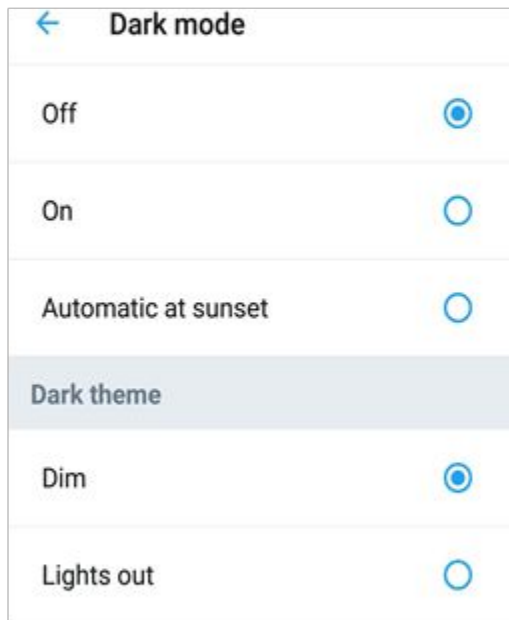




Res/values/Style.xml

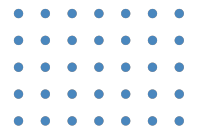


Twitter Dark mode On / Off



```
<manifest ... >
  <application
    android:theme="@style/Theme.AppCompat"...>
  </application>
</manifest>

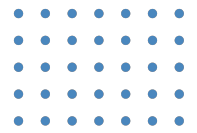
<manifest ... >
  <application ... >
    <activity
      android:theme="@style/Theme.AppCompat.Light"
    ... >
    </activity>
  </application>
</manifest>
```



Res/values/Style.xml

Name

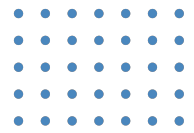
```
<style name="PurpleText"
parent="TextAppearance.AppCompat">
    <item
name="android:textColor">#7700FF</item>
    <item name="android:textSize">20sp</item>
</style>
```



Dimens

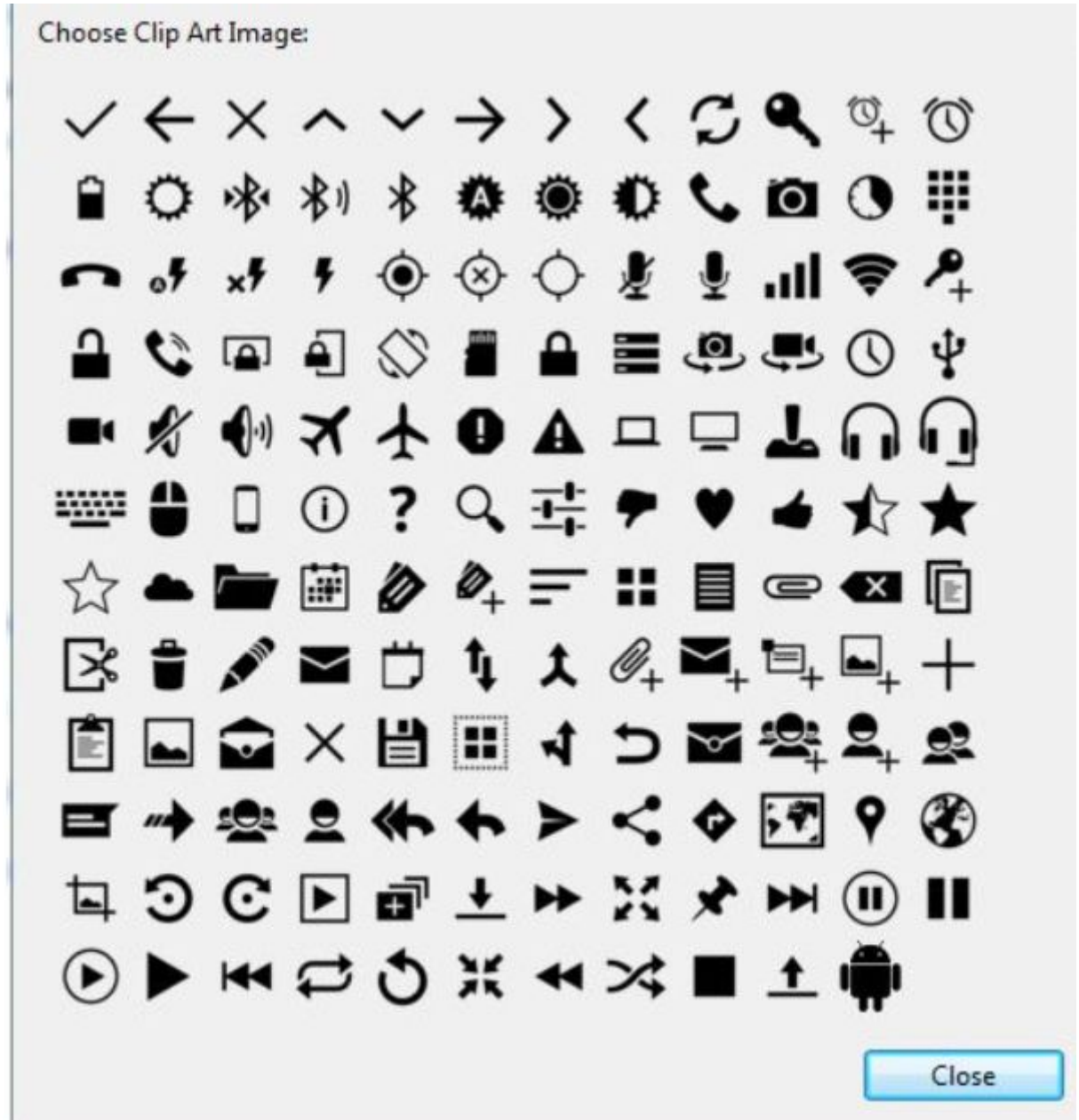
Dentro del diseño de aplicaciones, el tamaño de los objetos es un elemento crucial. Es recomendable mantener estos tamaños separados de los estilos para un control centralizado. Esto se logra a través de un archivo llamado `dimens.xml` ubicado en la carpeta `res/values` junto con `styles.xml`. Separar los tamaños permite un mejor control sobre las dimensiones de la aplicación durante el desarrollo y a ser consistente en las dimensiones entre pantallas

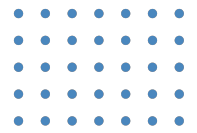
```
ml>  
en">  
ead>  
  
app">  
plication>  
v-root (open)  
rel="stylesheet" type="text/css" href="//  
n.bootstrapcdn.com/bootstrap/4.0.0-beta.2/css/  
trap.min.css">  
...</style>  
class="navbar navbar-expand-md navbar-dark bg-dark">  
class="container">  
-form>  
yle>...</style>  
/ class="card todo-form">...</div>  
-form>  
  
list ref="list">  
e>...</style>  
asks:</h2>  
ef="todos" class="list-group">  
o-task ref="task-1517176192142" id="task-151717619  
o-task> == $0  
o-task ref="task-1517176320397" id="task-151717632  
o-task>  
-task ref="task-1517176329096" id="task-1517176329  
-task>  
-task ref="task-1517176334849" id="task-1517176334  
-task>  
t>  
nn>
```



Drawables

El directorio "drawable" se usa para almacenar recursos gráficos como iconos o imágenes en una aplicación. Puede contener subdirectorios como drawable-mdpi, drawable-hdpi, drawable-xhdpi, entre otros, para diferentes densidades de píxeles de dispositivos Android. Android selecciona automáticamente el recurso gráfico adecuado según la densidad de píxeles del dispositivo. Para acceder a estos archivos, se utiliza la sintaxis @drawable/nombre_del_recurso, y estos recursos se encuentran en el directorio res.





Raw

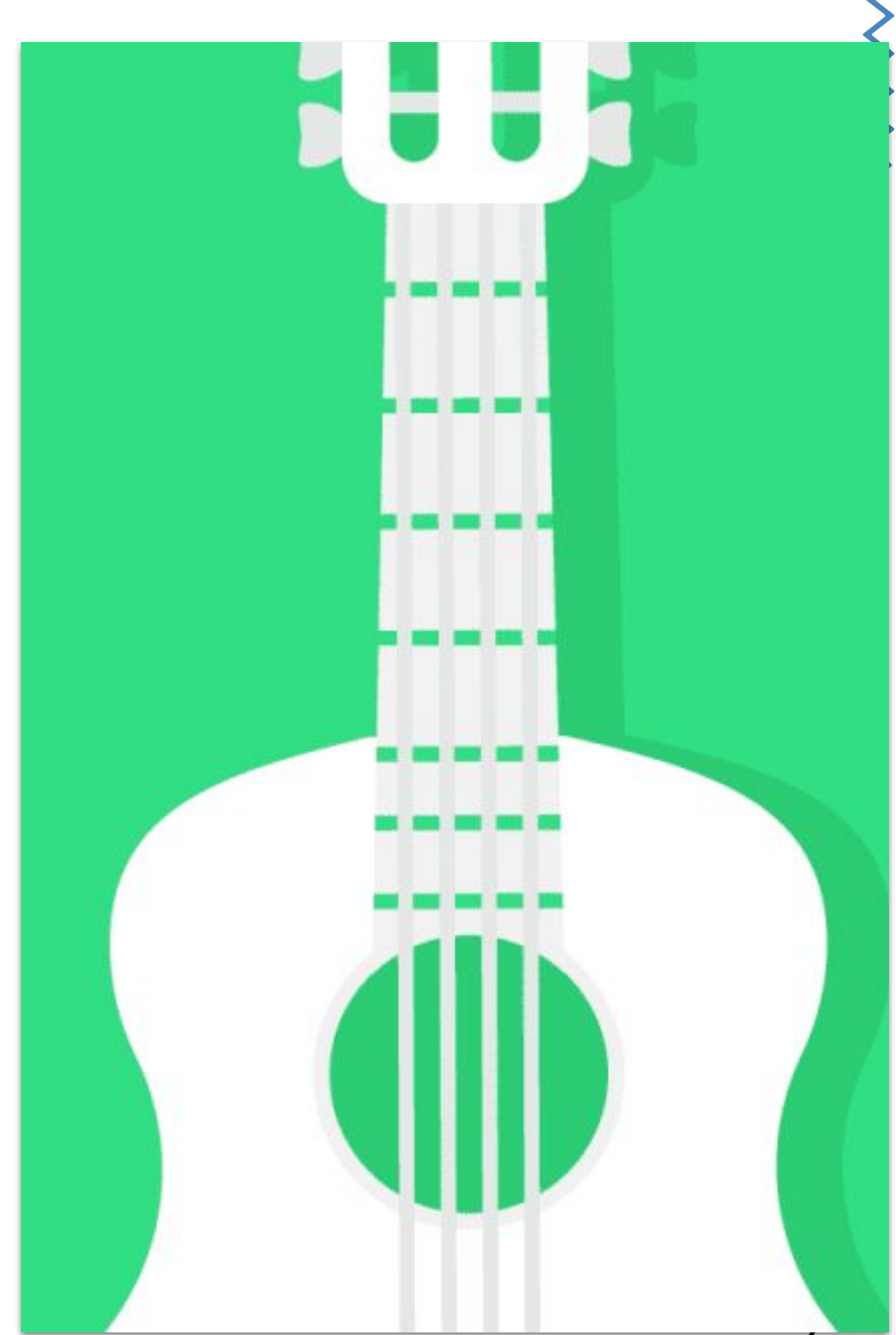
El directorio "raw" en Android es un lugar especial para almacenar archivos de recursos sin procesar, como canciones en MP3, videos en MP4, archivos binarios o de texto, e incluso archivos HTML o XML. Estos archivos se mantienen sin cambios y se almacenan tal como están, sin compilación ni procesamiento. La ventaja principal es que se pueden acceder directamente desde el código utilizando simplemente su nombre, sin necesidad de procesamiento adicional.





Resources

Los recursos en Android son esenciales para personalizar y proporcionar contenido estático en una aplicación. Facilitan la adaptación a diferentes dispositivos y preferencias de usuario al gestionar elementos como estilos, textos e imágenes de manera eficiente y organizada. En este directorio van a poder encontrar varios recursos como Strings (Con todo el texto de la app) y la declaración de colores en colours



LIVE CODING

Subtítulo: Recursos

Revisión de directorios en un nuevo proyecto

1. Ver el Manifest
2. Crear Assets
3. Explorar Styles
4. Ver todos los Drawables
5. Agregar a Raw
6. Abrir los recursos por defecto en res

Tiempo: 15 Minutos

Scripts de compilación



Scripts de compilación

¿Qué es un script?:

Un script de compilación en Android es un archivo que guía al sistema sobre cómo construir nuestra aplicación y qué elementos debe incluir en ella. Es esencial para el desarrollo de aplicaciones Android, ya que asegura que la aplicación funcione correctamente en dispositivos. En Android, este script se utiliza con la herramienta Gradle para generar un archivo APK de instalación que se ejecuta en dispositivos Android.

Scripts de compilación

Scripts:

Existen múltiples Scripts que funcionan en conjunto para compilar la aplicación algunos de ellos son:

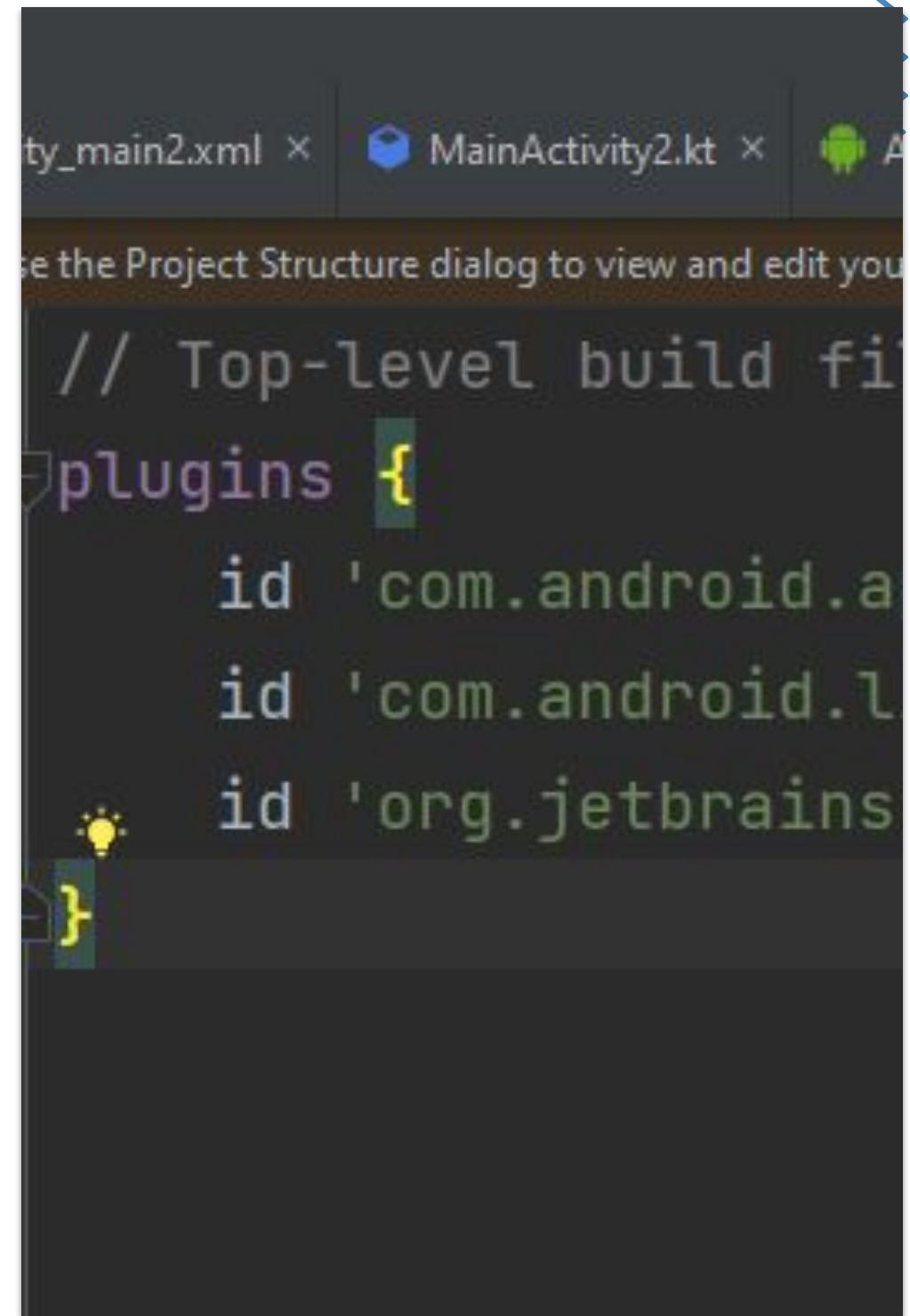
- build.gradle (módulo de la aplicación)
- build.gradle (proyecto)
- settings.gradle
- gradle.properties
- proguard-rules.pro

```
plugins {  
    id 'com.android.application'  
    id 'org.jetbrains.kotlin.android'  
}  
  
android {  
    namespace 'com.example.myapplication'  
    compileSdk 33  
  
    defaultConfig {  
        applicationId "com.example.myapplication"  
        minSdk 24  
        targetSdk 33  
        versionCode 1  
        versionName "1.0"  
  
        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"  
    }  
  
    buildTypes {  
        release {  
            minifyEnabled false  
            proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-rules.pro'  
        }  
    }  
}
```

Scripts de compilación

Gradle Proyecto:

Para el nivel del proyecto, encontrarás el archivo `build.gradle` en la carpeta raíz del proyecto. Este archivo contiene la configuración general del proyecto, como la configuración de Gradle y las dependencias del proyecto en su conjunto. También puedes definir propiedades globales que se pueden utilizar en otros scripts de Gradle dentro del proyecto. Esto es útil para mantener variables compartidas. Es posible al mismo tiempo posible establecer configuraciones globales que afecten a todos los módulos de la aplicación, como las configuraciones de ProGuard, las versiones de bibliotecas compartidas y más.



```
// Top-level build file
plugins {
    id 'com.android.a'
    id 'com.android.l'
    id 'org.jetbrains'
```

Scripts de compilación

Gradle App:

En Android, el archivo Gradle específico del módulo de la aplicación (conocido comúnmente como "build.gradle (app)") se utiliza para configurar y controlar aspectos relacionados con la construcción y el funcionamiento de la aplicación específica en ese módulo. Algunos de los usos más comunes de este archivo incluyen:

```
plugins {  
    id 'com.android.application'  
    id 'org.jetbrains.kotlin.android'  
}  
  
android {  
    namespace 'com.example.myapplication'  
    compileSdk 33  
  
    defaultConfig {  
        applicationId "com.example.myapplication"  
        minSdk 24  
        targetSdk 33  
        versionCode 1  
        versionName "1.0"  
  
        testInstrumentationRunner "androidx.test.runner.AndroidJUnitRunner"  
    }  
  
    buildTypes {  
        release {  
            minifyEnabled false  
            proguardFiles getDefaultProguardFile('proguard-android-optimize.txt'), 'proguard-rules.pro'  
        }  
    }  
}
```

LIVE CODING

Subtítulo: Revisar Gradle

Revisar las propiedades generales de el archivos Gradle App e identificar las siguiente propiedades:

1. *Plugins definidos*
2. *El targetSDK*
3. *El compileSDK*
4. *El minSDK*
5. *Las dependencias de implementación*
6. *Las dependencias de Testing*
7. *Las dependencias de Testing de Android*

¿Alguna consulta?





¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

